PROJECT GUIDE · INTELLIGENT CANDIDATE DISCOVERY & RANKING

The Redrob Candidate Ranker

An AI system that ranks job candidates the way a great recruiter would — by understanding what a role actually needs, not by matching keywords. This guide explains the idea from scratch, why it matters in the real world, the technology, the pipeline, the project structure, and exactly how to run it.

100,000	Top 100	~37 s	Hybrid
candidates ranked	trusted shortlist + reasons	on a normal laptop CPU	meaning + rules + signals

What's inside

SECTION 1

The core idea — from scratch

A company posts one job. Thousands of people are in the talent pool. A recruiter can only personally read a handful. So how do you find the **few people who genuinely fit** — fast, fairly, and without missing the quiet gems?

The problem with the old way

Most hiring tools work by **keyword matching**: the job says "RAG, Pinecone, embeddings", so the tool returns everyone who typed those words on their profile. This breaks in two directions:

- **False positives** — a Marketing Manager who took an online course and listed "RAG, Pinecone, FAISS" looks like a perfect match to a keyword filter, but cannot do the job.
- **False negatives** — the best engineer may describe their work in plain English ("I built the systems that decide what users see") and *never use the buzzword* — so a keyword filter skips them entirely.

The insight this project is built on

> A job description **says one thing but means another**. The real signal of fit is not the words in a skills list — it is **what a person actually built** (their career history), **whether they're a genuine fit** for the role's intent, and **whether they're actually reachable and available**. Buzzwords are noise; evidence and availability are signal.

What this system does, in one sentence

It reads every candidate's full profile, **understands** it (using AI language models that grasp meaning, not just words), cross-checks it against a structured model of what the role truly needs, **throws out the fakes and impossible profiles**, accounts for who is actually available, and returns a ranked shortlist of the top 100 — each with a short, honest reason.

✕ A keyword-stuffer

"Project Manager · AI enthusiast." Skills list full of RAG, Pinecone, FAISS. Summary: "took online courses on RAG."
→ our system scores it ~0.04 (buried)

✓ A real fit in plain words

"Senior AI Engineer who built the ranking and retrieval systems that decide what to show." Never says "RAG".
→ our system scores it ~0.80 (top)

SECTION 2

Why it's useful in the real world

This is not just a contest entry — it is a blueprint for the "intelligence layer" of any modern hiring or talent product.

Where it helps	What it does in practice
Recruiting platforms & job boards	Powers the search that decides which candidates a recruiter sees first — surfacing genuine fits instead of the loudest keyword profiles.
Applicant tracking (ATS)	Auto-ranks incoming applicants for a role so recruiters spend their limited time on the most promising people.
Sourcing / headhunting	Scans a large pool and pulls out hidden gems who describe their work in plain language and would be missed by keyword tools.
Internal mobility	Matches existing employees to new internal openings based on what they've actually done.

The concrete benefits

- **Saves time** — turns thousands of profiles into a trustworthy top-100 in seconds.
- **Finds hidden talent** — semantic understanding catches strong people who don't self-market.
- **Filters fakes** — rejects buzzword-stuffers and logically impossible profiles automatically.
- **Explainable & auditable** — every pick comes with a reason and every score can be traced, which matters for trust and fair-hiring compliance.
- **Cheap & scalable** — runs on an ordinary CPU with no paid AI API calls in the loop, so it scales to hundreds of thousands of profiles.
- **Respects availability** — won't push someone who hasn't logged in for 6 months to the top.

Who wins: recruiters (less noise), hiring managers (better shortlists), companies (faster, cheaper hiring), and candidates (a fairer shot judged on real evidence, not buzzword bingo).

SECTION 3

How it works — the pipeline

There are two phases. A one-time **offline** phase prepares the AI understanding of every profile. Then the fast **ranking** phase (the part you run for each job) scores everyone and writes the shortlist. No internet, no GPU, and no per-candidate AI API calls are needed during ranking.

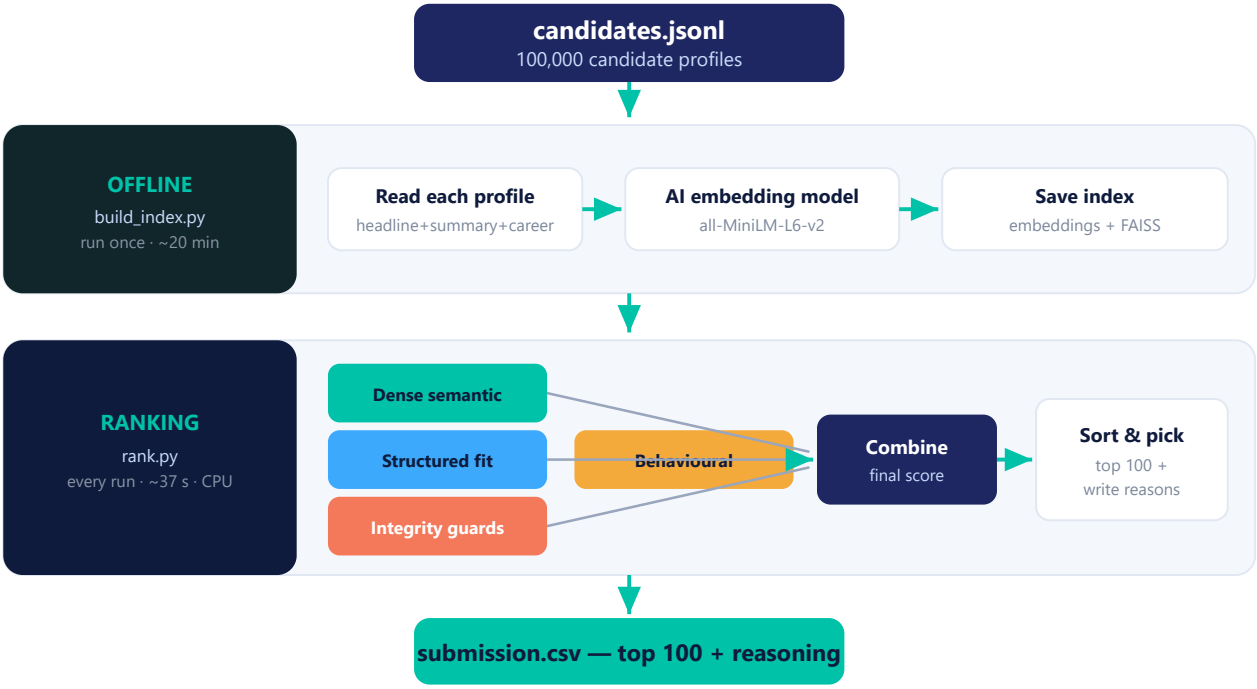


Figure 1 — End-to-end pipeline: a one-time offline embedding step, then a fast CPU-only ranking step.

Step by step

- ① **Offline — understand every profile (once).** `build_index.py` feeds each profile through a small AI language model (*all-MiniLM-L6-v2*) that turns text into a 384-number "meaning fingerprint" (an *embedding*). These are saved so ranking never has to recompute them.
- ② **Load.** `rank.py` streams all 100,000 profiles into memory.
- ③ **Score on four lanes** (explained in Section 4): **meaning** **fit rules** **fraud guards** **availability**.
- ④ **Combine** the lanes into one final score per candidate.
- ⑤ **Sort & shortlist** — take the top 100 and write a specific, honest reason for each.

SECTION 4

How a candidate is scored

Each candidate's final score is a "fit" score multiplied by a series of reality-check multipliers. The multipliers are what keep traps out of the shortlist.

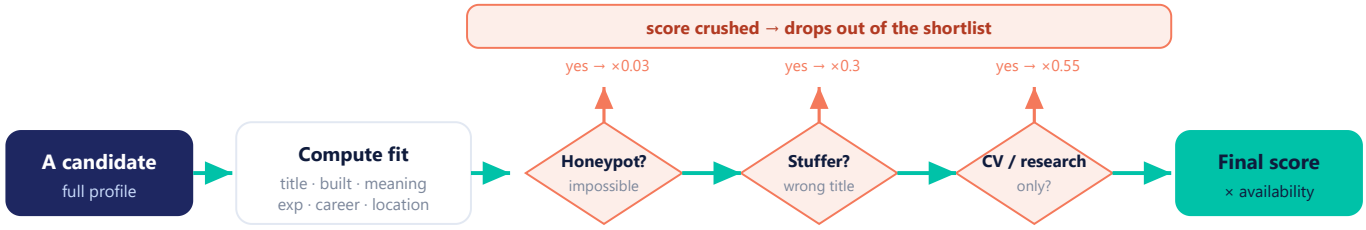
```
# the master formula
final = fit  ×  behavioural  ×  disqualifier  ×  honeypot  ×  stuffer
```

"fit" = a weighted blend

Signal	Weight
Title (current + career)	0.22
What they actually built	0.22
Career arc (product vs services)	0.14
Meaning match — retrieval focus	0.13
Meaning match — full job	0.11
Experience band (5–9 yrs)	0.10
Location	0.08

The multipliers (reality checks)

Check	Effect
Behavioural availability	× 0.62 – 1.08
Computer-vision-only, no retrieval	× 0.55
Research-only, no production	× 0.60
Keyword-stuffer	× 0.25 – 0.40
Honeypot (impossible profile)	× 0.03



A clean, genuine candidate passes every check and keeps a high score; a trap gets multiplied down to near-zero.

Figure 2 — How one candidate is scored: fit first, then reality-check multipliers that filter traps.

Why this beats keyword matching

- **Credit for AI skills is "gated" by credibility** — a buzzword in a Marketing Manager's skills list earns almost nothing; the same word on a genuine engineer earns full weight.
- **Honeypots are caught by simple logic** — e.g. "8 years at a company that's 3 years old", or "expert in 10 skills used for 0 months". These get multiplied to ×0.03 and disappear.
- **Meaning, not words** — the AI embedding recognises "ranking and retrieval systems that decide what to show" as a perfect match even though it never says "RAG" or "Pinecone".

SECTION 5

Technology stack

Deliberately lightweight and free: standard open-source Python libraries, a small local AI model, and no paid APIs anywhere in the ranking path.

Technology	Role	Why it's used here
Python 3	Core language	The whole system; clean, readable, easy to audit.
sentence-transformers all-MiniLM-L6-v2	AI "understanding"	Turns each profile into a meaning-vector so we can match by intent, not keywords. Small & CPU-friendly.
PyTorch (CPU)	Runs the model	The engine under the embedding model; CPU-only build, no GPU needed.
FAISS	Vector index	Facebook's similarity-search library — the "vector database" path for fast retrieval.
NumPy	Math & scoring	All the scoring is fast array math — this is why ranking takes seconds, not minutes.
scikit-learn	Fallback search	TF-IDF backup so the system still runs even without the embedding model present.
Streamlit	Demo web app	The interactive "sandbox" where you upload a sample and see it ranked live.
python-pptx	Docs	Generates the approach slide deck programmatically.

Key design choice: the heavy AI work (embedding 100K profiles) happens **once, offline**. The part you run for each job is pure fast math over the saved index — which is why it meets a strict "5 minutes, CPU-only, no internet" production budget.

The "hybrid" in plain terms

Dense / semantic side

The AI model gives **recall** — it finds good people even when they don't use the expected words.

Structured / rules side

The hand-built fit model + guards give **precision** — they keep fakes, wrong-domain, and unavailable people out of the top.

SECTION 6

Project structure

Every file has one clear job. The `src/` folder holds the brain; the top-level scripts are what you actually run.

```
C:\hackathon\
├─ rank.py                 # ► MAIN: produces the top-100 submission.csv
├─ build_index.py          # offline: builds the AI embedding index (run once)
├─ app.py                  # Streamlit demo / sandbox web app
├─ validate_submission.py  # official format checker
├─ requirements.txt         # the libraries to install
├─ README.md                # setup + reproduce instructions
├─ submission_metadata.yaml # team / approach declaration
│
├─ src\                    # the scoring brain
│  ├─ jd_spec.py            # the job description distilled into rules & keywords
│  ├─ profile.py            # reads a candidate, builds its text + fields
│  ├─ semantic.py           # the AI meaning-match (embeddings + fallback)
│  ├─ features.py           # structured fit: title, what-they-built, experience…
│  ├─ integrity.py          # honeypot + keyword-stuffer detection
│  ├─ behavioral.py         # 23 platform signals → availability multiplier
│  ├─ reasoning.py          # writes the honest 1-2 sentence reason per pick
│  ├─ scoring.py            # blends everything into the final score
│  └─ ranker.py             # orchestrates: load → score → sort → write
│
├─ artifacts\              # pre-computed (built by build_index.py)
│  ├─ embeddings.f16.npy    # the 100K meaning-vectors
│  └─ candidate_ids.npy / jd_vectors.npy
│
├─ docs\                   # approach_deck.pdf + this project_guide.pdf
└─ tests\                   # trap-defense sanity checks
```

Mental model: `rank.py` is the button you press. It calls `src/ranker.py`, which uses every other `src/` file as a specialist (meaning, fit, fraud, availability, reasoning) and writes `submission.csv`.

SECTION 7

How & where to run it

Where it runs (platforms)

Platform	Status
Windows / macOS / Linux	✅ Any laptop or desktop with Python 3 — this is the normal way.
Hardware needed	✅ Ordinary **CPU**, ~16 GB RAM. **No GPU and no internet** needed for ranking.
Cloud notebooks	✅ Google Colab / Jupyter / Binder — clone and run the same commands.
Hosted sandbox	✅ `app.py` deploys to Hugging Face Spaces or Streamlit Cloud (free tiers).
Docker	✅ Runs in any standard Python container.

The commands (Windows PowerShell)

Open PowerShell and go to the project. Set the path to the candidate file once so you don't retype it:

```
# 0) go to the project + point at the data file
cd C:\hackathon
$cands = "C:\Users\akash\Downloads\[PUB] India_runs_data_and_ai_challenge\[PUB]
India_runs_data_and_ai_challenge\India_runs_data_and_ai_challenge\candidates.jsonl"
```

▶ **Normal run** — the embeddings are already built, so this just produces the shortlist (~37 s):

```
# 1) produce the ranked top-100
python rank.py --candidates $cands --out submission.csv

# 2) check the file is valid
python validate_submission.py submission.csv
```

↻ **Full rebuild from scratch** — only if you want to regenerate the AI index (~20 min, one time):

```
pip install -r requirements.txt
python build_index.py --candidates $cands        # builds artifacts\ (offline)
python rank.py --candidates $cands --out submission.csv
```

📱 **Interactive demo** — opens a web app at `http://localhost:8501` to rank a sample live:

```
streamlit run app.py
```

If you move the project or use a different data location, just change the `$cands` path. On macOS/Linux the commands are identical except you'd write `cands="/path/to/candidates.jsonl"` and use `$cands` the same way.

SECTION 8

What it produces & quick reference

The output: `submission.csv`

Exactly 100 rows — best fit first — with four columns:

candidate_id	rank	score	reasoning
CAND_0018499	1	0.9900	Senior ML Engineer with 7 yrs (Zomato, Google); direct ranking & embeddings experience…
CAND_0064326	2	0.7962	Search Engineer with 8 yrs (Sarvam AI); direct ranking + recommendation systems experience…
CAND_0002025	3	0.7901	Senior AI Engineer with 6 yrs (Apple); direct ranking + recommendation experience…

Proof it works (on the released pool)

100%
engineering titles in top 100

86/100
in the ideal 5–9 yr band

0
honeypots & 0 fakes in top 100

37 s
CPU only, no internet

One-glance cheat sheet

I want to…	Run this
Produce the shortlist	`python rank.py --candidates $cands --out submission.csv`
Check the output format	`python validate_submission.py submission.csv`
Rebuild the AI index	`python build_index.py --candidates $cands`
Open the live demo	`streamlit run app.py`
Install dependencies	`pip install -r requirements.txt`

In one line: this project reads 100,000 résumés, understands them with AI, weeds out the fakes, respects who's actually available, and hands a recruiter a trustworthy, explained top-100 — in under a minute, on a normal laptop, with no paid AI calls.